

Final Project Proposal

Prediction to Prescription: Building an AI Agent

Team: 7 **Project Title:** Uniform Ordering Optimization for a Youth Soccer League

Date: May 31st, 2026

Name	Email	Program	Primary Role
Satya Sai Usha Sree Ganteda	sganteda@purdue.edu	MS Mec.Eng.	Problem Formulation Module
Daniel Tulloch Affleck	daffleck@purdue.edu	MBA	Data Collection & Preparation
Siddharth Shenoy	shenoy22@purdue.edu	MSBAIM	Prediction Layer
Fabian Alexis Macias Peñalosa	fmaciasp@purdue.edu	MSBAIM	Optimization Engine
Aditya Ryan Gupta	gupta877@purdue.edu	MSBAIM	Explanation & Validation

Section 1 — Problem Statement and Business Context

1.1 Business Context

CFA (Club Futbol Academy) is a youth soccer league in Orange County, California, running four seasonal programs (fall, winter, spring, and a year-round competitive division). Each season the league president must commit to uniform order quantities — across jerseys, shorts, and socks in up to 15 size codes — before registration demand is observed, processing roughly 2,000–3,000 orders per season through its Squarespace storefront.

Order quantities are currently set by 25 years of operational intuition. This heuristic produces systematic errors in both directions: overstock ties up capital in inventory children outgrow, while stockouts trigger rush orders with premium shipping and delayed fulfillment.

The problem is compounded by Adidas’s jersey lifecycle. Each jersey model (e.g., Condivo 22, Tiro 25) is produced for approximately two years before being discontinued and replaced. This creates a **two-period decision structure** with escalating costs:

- **Year 1 overage:** Unused jerseys carry to Year 2 at minimal holding cost.
- **Year 1 underage:** Rush orders at the same unit price but with added shipping costs and fulfillment delays.
- **Year 2 overage:** Excess inventory is sold on eBay at a salvage price, incurring a loss.
- **Year 2 underage:** The jersey model is discontinued by Adidas. Sourcing secondhand stock requires significant premiums, or the sale is lost entirely — resulting in lost revenue and unhappy parents.

1.2 Problem Class

This is a **multi-period, multi-product newsvendor ordering problem**. Each season the league decides how many units of each size to order for three product categories before demand is realized. The goal is to minimize total procurement cost — holding, rush-order premiums, salvage losses, and stockout penalties — subject to a seasonal budget and supplier minimum order quantities. The two-year jersey lifecycle adds a temporal coupling: Year 1 decisions directly affect Year 2 feasibility and cost through carryover inventory.

1.3 Why This Problem Is Hard Without an Agent

Three factors make this problem difficult without algorithmic support. First, **demand is non-stationary**: youth players grow between seasons, so last year’s size distribution is a biased predictor of next year’s. Second, **the cost structure is asymmetric and escalating**: minimizing MSE is not the same as minimizing decision cost (Bertsimas–Kallus), and in Year 2 the underage penalty can be an order of magnitude larger than the overage penalty. Third, **decisions are combinatorially coupled**: every size order shares the same budget, so the optimal quantity for one size depends on every other simultaneously. We expect the agent to outperform the current heuristic by learning the cost-asymmetric demand distribution from six years of order history and recommending size-level quantities that minimize expected total cost rather than prediction error.

Section 2 — Data Plan

2.1 Primary Dataset

Field	Details
Dataset name	CFA Squarespace Order History
Source / URL	Exported from the league’s Squarespace e-commerce backend (proprietary; use authorized by league president)
Approximate size	90,010 rows $\times$ 52 columns; August 2017 – May 2026
Key features	Order date, product name, category (top/bottom/socks), size variant, quantity, unit price, uniform set, gender/age group
Outcome variable	Demand quantity per size $\times$ product category $\times$ season
Known limitations	Size embedded in free-text variant string (requires parsing); product names inconsistent across seasons (requires normalization); lifecycle year must be inferred from product name and order date

After cleaning: **12,209 official game uniform line items** spanning 2020–2026, across 15 normalized size codes, 4 uniform sets, and 3 product categories. Player names have been removed for privacy. The cleaning pipeline is reproducible via `scripts/clean_data.py` in the project repository.

2.2 Additional Datasets

Registration data (pending): player-level age, gender, and team records to enrich size-demand prediction by connecting demographic features to sizing patterns. The league president has agreed to provide this data. **Adidas lifecycle data**: release and discontinuation dates for Condivo 20/22 and Tiro 25/26 models, sourced from public retailer databases, used to label each order by lifecycle year.

2.3 Train / Test Split

Chronological split: **train on 2020–2024** (approximately 9,500 line items across 8–10 seasons), **test on 2025–2026** (approximately 2,700 line items across 4 seasons). This preserves temporal ordering and prevents data leakage from future seasons into training. The test set includes seasons from both Year 1 and Year 2 of the current jersey lifecycle, enabling evaluation of the agent’s performance under both cost regimes.

Section 3 — Methods Sketch

3.1 Prediction Layer

We compare three approaches with increasing decision-awareness (Bertsimas–Kallus, Lectures 2 and 4):

- (1) **PTO-Naive** — MSE regression, predicted demand used directly as the order quantity (the “wrong loss” baseline);
- (2) **PTO-Adjusted** — same MSE regression but order shifted by the newsvendor critical ratio $q^* = \hat{D} + \sigma \cdot \Phi^{-1}(\frac{c_u}{c_u + c_o})$, partially correcting for cost asymmetry;
- (3) **End-to-End Prescriptive** — a linear model trained directly on newsvendor cost via Adam, with the gradient flowing back through the chain rule

$$\frac{\partial \mathcal{L}_{\text{NV}}}{\partial \beta} = \underbrace{\frac{\partial C}{\partial q}}_{\pm c_o / -c_u} \cdot \underbrace{\frac{\partial q}{\partial \hat{D}}}_{e^{\sigma \Phi^{-1}}} \cdot \underbrace{\frac{\partial \hat{D}}{\partial \beta}}_{\hat{D} \mathbf{x}}$$

allowing the model to develop intentional biases (e.g., over-ordering in Year 2 where underage costs dominate). The Data Scientist Agent selects the best performer on holdout newsvendor cost, not RMSE.

3.2 Optimization Engine

The constrained multi-product newsvendor is solved via `scipy.optimize.minimize` (SLSQP), minimizing the SAA objective:

$$\min_{\mathbf{q}} \frac{1}{N} \sum_{i=1}^N \sum_s [c_o^{(s)}(q_s - D_{s,i})^+ + c_u^{(s)}(D_{s,i} - q_s)^+]$$

subject to budget, minimum order, and non-negativity constraints. To handle potential non-convexity in the multi-product case we use 15+ randomized restarts. The LLM-based agent generates solver code from the `ProblemConfig`, adapting c_o/c_u by lifecycle year and product category. KKT shadow prices quantify the business cost of each binding constraint.

3.3 Constraint Handling & Baseline

Constraints are passed as **hard constraints** to SLSQP; a projection step maps any violated solution back to the feasible set. The baseline is a **fixed-order policy** — the per-size average across training seasons — approximating the president’s current heuristic. The agent is evaluated against this baseline on total season newsvendor cost (procurement + rush shipping + salvage + stockout), not prediction accuracy.

3.4 Pipeline Diagram

User (NL) → Problem Formulator → ProblemConfig → Data Prep → Data Scientist Agent
(PTO-Naive / PTO-Adjusted / End-to-End NV | SLSQP + SAA) → Reporter Agent → Dashboard +
PDF + Shadow Prices

Each agent writes structured JSON to a shared run directory; subsequent agents load prior outputs with no direct coupling. All steps are logged for audit and explainability.

Section 4 — Team Roles

Component	Primary Owner	Brief Description of Responsibility
Problem Formulation Module	Satya Ganteda	NL intake agent; ProblemConfig schema; benchmark test integration
Data Collection & Preparation	Daniel Affleck	Cleaning pipeline, lifecycle labeling, feature engineering, train/test split
Prediction Layer	Siddharth Shenoy	PTO and prescriptive NV-loss models; holdout cost comparison
Optimization Engine	Fabian Macias	SLSQP solver, multi-start restarts, constraint handling, shadow prices
Explanation & Validation	Aditya Gupta	Reporter agent, sensitivity analysis, dashboard, PDF export

All members collaborate across components; the cleaning pipeline is largely complete. Satya and Fabian co-own optimization formulation; Daniel and Siddharth run a joint integration checkpoint before the optimizer receives inputs.

Section 5 — Benchmark and Validation Plan

5.1 Evaluating Your Own Problem

The primary metric is **total season newsvendor cost** on the 2025–2026 holdout:

$$\text{Total Cost} = \sum_s [c_o^{(s)}(q_s - D_s)^+ + c_u^{(s)}(D_s - q_s)^+]$$

where c_o/c_u vary by product and lifecycle year. We report: (1) average season cost reduction (%) vs. the fixed-order baseline; (2) stockout and overstock rates per size; and (3) incremental gains from PTO-Adjusted over PTO-Naive and from End-to-End Prescriptive over PTO-Adjusted. A 10–15% cost reduction would be practically meaningful, representing hundreds of dollars per season.

5.2 Stakeholder Robustness

What is parameterized: The full problem definition lives in a `ProblemConfig` JSON schema — decision variables, objective, constraints, and cost structure (c_o , c_u per product and lifecycle year). The Problem Formulator Agent parses natural-language modifications into this config; any field can be reviewed and edited before the pipeline re-runs.

Handled without re-engineering: (1) A new constraint (e.g., “add a sustainability cap of 500 total units”) → add to `constraints` list, re-run optimizer. (2) A shifted objective (e.g., “maximize service level instead of minimizing cost”) → update `objective` field, regenerate solver code. (3) Changed costs (e.g., “rush shipping now costs $2\times$ more”) → update `cost_structure`, re-run optimizer and explanation. (4) A new decision variable (e.g., “also decide how many number transfers to pre-order”) → add to `decision_variables`, may trigger solver code regeneration.

Limits of adaptability: The agent handles any change expressible in `ProblemConfig` fields. A fundamentally different problem class (e.g., switching from inventory ordering to staff scheduling) or a new data source not present in the existing dataset would require re-engineering of the data and prediction modules. We will be transparent about this boundary during the Q&A.

References

- [1] Bertsimas, D. & Kallus, N. “From Predictive to Prescriptive Analytics.” *Management Science*, 66(3), 1025–1044, 2020.
- [2] Kingma, D.P. & Ba, J. “Adam: A Method for Stochastic Optimization.” *ICLR*, 2015.
- [3] Nocedal, J. & Wright, S. *Numerical Optimization* (2nd ed.). Springer, 2006.
- [4] Jaiswal, P. “From Prediction to Prescription: Why Optimization Projects Fail in Practice.” *MGMT 590-037 Lecture Notes*, Summer 2026.
- [5] Jaiswal, P. “SGD as a Learner: From Prediction to Prescription — Closing the Loop.” *MGMT 590-037 Lecture Notes*, Summer 2026.

Submit: Push `proposal.pdf` to your team GitHub repo and share the link with `jaiswalp@purdue.edu`
MGMT 590-037 — Project Proposal Template — Summer 2026